

Text OCR

Wednesday, July 15, 2026 7:35 PM

[TextInPlace: Indoor Visual Place Recognition in Repetitive Structures with Scene Text Spotting and Verification \(March 2024\)](#)

Text OCR is used to the initial VPR matches:

Text recognized via DPText OETR (transformer model specific to bounding).

DIG extracts the text.

GPT-4o determines if the text is meaningful or not. No embedding. < This is an optional addition,

The DPText OETR -> DIG pipeline is specialized for curly, oblique, hard to recognize text. It is resource intensive, minus the LLM, it accounted for 95% of the processing time. 170 ms out of ~ 200 ms.

Reranking is only slow when the LLM is incorporated:

TABLE VI: Comparison of computational cost.

Method	Latency per Query (ms)			
	Extrac.	Retriev.	STS	Re-rank
BoQ + DG-Bridge Spotter	7.7	2.9	165.7	0.1
TextInPlace	5.9	2.9	161.0	0.1
TextInPlace-L	5.9	2.9	161.0	962.8

Text OCR incorporation into VPR is effective but the much slower than the CV models used (ex: DINO, RESNET)

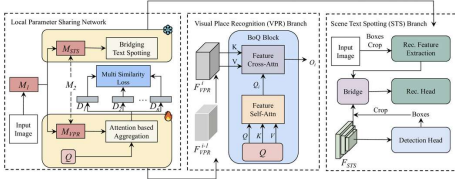


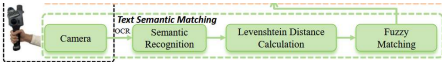
Figure 2: An overview of the TextInPlace framework. TextInPlace is (a) a local parameter sharing network with two branches. (b) The Visual Place Recognition Branch extracts global descriptors for coarse global retrieval. (c) The Scene Text Spotting Branch detects and recognizes scene text in query and database images. Finally, re-ranking is performed based on the text similarity between the query image and the retrieved images.

TextSLAM < Tale notes on this! Recommendations on limiting the amount of runs needed



Show All

[TWC-SLAM: Multi-Agent Cooperative SLAM with Text Semantics and WiFi Features Integration for Similar Indoor Environments \(Oct 2023\)](#)



Limitations:

Alternative OCR Pipelines:

STRIDE < text extraction from bounding box

Or:

[PP-OCRv5 Mobile Detector -> planar tracking/verification -> PP-OCRv5 Mobile Recognizer]

38 ms on a 14 GPU or 28.15 ms on the paper's test CPU

4.7 MB and 5 million parameters for the complete system

[Context-Free TextSpotter for Real-Time and Mobile End-to-End Text Detection and Recognition](#)

Also a strong mobile model. Made for resource efficiency.

Semantic/Text only slam cannot do pose estimation but it can do coarse segment matching which is sufficient. Rather than mapping to pose localize to individual rooms.

Potential Issues:

Semantic classification is stable, but text is not.

The label on the door may be read as 214 214 214 214

Potential Solutions:

[Vmethod: Robust Localization using Navigation Signs and Public Maps \(Aug 2023\)](#)

OCR and parsing errors by comparing an observed label against map labels using **normalized Levenshtein similarity**

Experimental results show that Vmethod reliably localizes the robot after observing only one to two signs.

Uses a VILM. Based extracting a map from OSM. Also dedicated towards robot navigation through the use of nodes

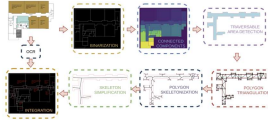


Figure 3: A diagram of the map extraction pipeline, starting with a venue map or a floor plan, and extracting the navigational graph. After the venue map is identified and the text and symbols are extracted, computational geometry methods are applied to convert the traversable nodes and the place nodes of the navigational graph. Then spatial and text information is integrated into the graph, connecting portal nodes and assigning labels to place nodes.

Observation Model. Given observed navigational cues $z_t = [z_t^1, \dots, z_t^T] = \{(l^1, D^1), \dots, (l^T, D^T)\}$, the observation model (Fig. 2) is:

$$p(z_t | x_t, m) = p(c_t | x_t, S) = \prod_{i=1}^T p(z_t^i | x_t, S) \quad (2)$$

Specifically, the observation model is the geometric mean of observation likelihoods for each $z_t^i = (l_t^i, D_t^i)$. Each likelihood models how likely it is for a location corresponding to l_t^i to be found in a given direction d_i with respect to the robot's current location at node $x_t \in V$ with heading $\theta_t \in [-\pi, \pi]$:

$$p(z_t^i | x_t, S) = p(l_t^i, D_t^i | (x_t, \theta_t), S) = \sum_{u \in V} p(l_t^i = \text{label}(u) | \sum_{l \in S} p(\text{toward}(u) - d | (x_t, \theta_t), S) \quad (3)$$

Due to ambiguous place names or parsing errors, the extracted label l_t^i may not match exactly with any node labels, or may match with multiple node labels in S . To handle this, we consider the top- k most similar node labels, i.e. $P_{\text{top-}k} = \arg\max_{u \in V} p(l_t^i = \text{label}(u))$, where $p(l_t^i = \text{label}(u))$ is a normalized fuzzy matching score based on Levenshtein distance [24].

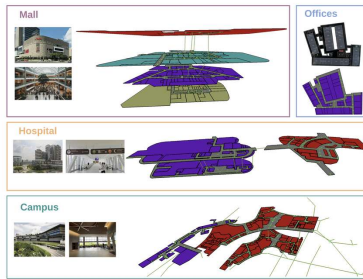


Figure 4: Qualitative results of our map extraction approach with multi-floor, multi-building navigational graphs extracted from publicly available maps. In the top right (offices) is our constructed navigational graph for the floor plan used by Xie et al. [40].

TABLE I: Evaluating sign understanding across 3 different environments.

	Cue Precision	Cue Recall	Sign Accuracy
Mail	0.824	0.847	0.902
Hospital	0.830	0.849	0.438
Campus	0.820	0.837	0.583

Additional IDEA: Guided movement with an interactive VILM based navigator that directs the user to go to certain locations for further information when localization is uncertain. This feels highly related to active SLAM

That matters because OCR has an unusually valuable property for localization: **exact or approximate symbolic identity**

```
"ROOM 214"  
"ROOM 21A"  
"214"  
"Rm 214"
```

Those strings can support edit distance, character-level similarity, numeric matching, token matching, etc. If you only embed them, you may throw away that explicit matching capability.

For your application, I would keep both:

```
sign node  
|--- semantic class: sign  
|--- OCR raw: "ROOM 214"  
|--- OCR normalized: "room 214"  
|--- text embedding: [...]   
|--- visual embedding: [...]   
|--- graph/context embedding: [...]
```

Then matching can combine:

$$S = w_p S_{\text{graph}} + w_v S_{\text{visual}} + \downarrow_{\text{text-embedding}} + w_o S_{\text{OCR-string}}$$